claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-a995b3e6a8e001862.jsonl`
- SHA-256: `fb4a1c54b8f927ed392c025d0313b1d1fbb89b47662196c9a1184f228dc4604b`
- Source modified: `2026-07-02T09:23:51+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_4fd1a4ea-6c0`
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`

Transcript

1. user (2026-07-02T09:20:08.330Z)

```
You are reviewing a god-module extraction for BEHAVIORAL DRIFT.
REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--
Users-avidu-Projects-badminton-highlight-
indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch
refactor/p23-orchestration-split off origin/master):
- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py:
  _finalize_reingest, _report_config_for_input, _ingest_remote_segments, _cloud_available,
  _maybe_dispatch_remote, _execute_processing, _remote_input_size_bytes, _CompileError,
  _resolve_compile, _execute_compile.
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration
import (...)" (noqa F401);
  5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):
  (1) _maybe_dispatch_remote: "req: ProcessRequest" -> string forward-ref "ProcessRequest"
(TYPE_CHECKING import).
  (2) _execute_processing: inserted "import backend.main as _main" after docstring;
global_config/db_instance/
      ProcessRequest resolved via _main.<name> at call time (the job_worker seam convention).
  (3) two "# type: ignore[...]" comments (AST-invisible).
- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity
check.
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green
(142 files),
  ratchet test passes. Full suite running separately.
Inspect with: git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff  (and
git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status); the new files are
untracked/modified in the worktree.
READ-ONLY: do not modify the worktree, do not checkout/commit.

The bodies are AST-identical; hunt for drift OUTSIDE the AST:
1. LOGGER NAME: moved code now logs under logger "backend.orchestration" instead of
"backend.main". Grep tests for caplog assertions (caplog.set_level, "in caplog.text",
record.name filters) that exercise the moved functions ? would any assertion break or silently
weaken? Also operational impact: log-analysis/observability docs or tools filtering on the
logger name "backend.main" for [COMPUTE]/[API]/[compile] lines emitted by moved code.
2. The _main.<attr> call-time resolution in _execute_processing: compare against master
semantics ? master read module globals of backend.main; new code reads attributes of the
backend.main module object at call time. Identical? Consider the python -m backend.main case
(module identity), pickling, and None-at-import-time.
3. ProcessRequest string forward-ref: anything that evaluates _maybe_dispatch_remote annotations
at runtime (FastAPI only evaluates ROUTE handler annotations; pydantic validate_call?
inspect.get_type_hints callers?).
```

```
4. The 2 type-ignore comments: confirm they don't mask a REAL latent bug worth noting (they were
latent in quarantined main too).
5. Coverage accounting: --cov=backend covers the new module automatically; any per-file coverage
config that lists main.py explicitly?
Report ONLY real, evidence-backed findings (file:line).
```

2. user (2026-07-02T09:20:16.584Z)

```
 M backend/main.py
?? backend/orchestration.py
?? tests/test_main_module_budget.py
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
 backend/main.py | 757 ++++----------------------------------------------------
 1 file changed, 50 insertions(+), 707 deletions(-)
```

3. user (2026-07-02T09:20:24.938Z)

```
"""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).

The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` / ``_ingest_remote_segments``),
and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is unchanged.

SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must remain
monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved through
the ``backend.main`` module object at call time (``import backend.main as _main``), never as a
bare local. ``backend.main`` re-exports every name below, so ``monkeypatch.setattr(backend.main,
...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working unchanged.
This module must NOT import ``backend.main`` at module level (main imports us for the re-
export).
"""

import csv
import json
import logging
import math
import os
import tempfile
from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
from urllib.parse import quote

from backend import storage
from backend.config import AppConfig, StitchingConfig
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id
from backend.utils.paths import output_base, safe_output_filename

if (
    TYPE_CHECKING
):  # runtime import would be circular (main imports us for the re-export)
    from backend.main import ProcessRequest  # noqa: F401

logger = logging.getLogger(__name__)


def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
```

```
    """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

    On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
    fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
    keep the prior good results intact ? a failed/empty re-run never destroys prior data.
    """
    if segments:
        db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
    else:
        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


def _report_config_for_input(config: Any, raw_input: str) -> Any:
    """Return report config that keeps remote-input reports out of disposable scratch."""
    storage_config = (
        config.get("storage")
        if isinstance(config, dict)
        else getattr(config, "storage", None)
    )
    try:
        if storage.input_is_local(raw_input, storage_config):
            return config
    except ValueError:
        return config
    try:
        cfg = (
            config.model_dump(mode="json")
            if hasattr(config, "model_dump")
            else dict(config or {})
        )
    except Exception:
        cfg = {}
    report = dict(cfg.get("report") or {})
    if report.get("output_dir"):
        return config
```

4. user (2026-07-02T09:20:36.631Z)

```
293:        storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy can't
narrow
539:            "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",  # type: ignore[arg-type]  # hasattr-unwrap above; Success arm has
.value
361:        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
364:        elif req is None and isinstance(config, _main.ProcessRequest):
369:        config = _main.global_config
371:        db = _main.db_instance
11:the ``backend.main`` module object at call time (``import backend.main as _main``), never as
a
13:...)`` and ``import backend.main as m; m._execute_processing(...)`` keep working unchanged.
355:    import backend.main as _main  # call-time seam: main's globals/model stay patchable
```

5. user (2026-07-02T09:20:44.366Z)

```
        return _failed(f"cloud job failed (returncode {result.returncode}).", True)
    if result.video_id and result.video_id != video_id:
        # The worker re-hashes the bucket bytes; a divergence means the input changed between
this
        # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
        logger.warning(
            "[API] cloud video_id %s != local %s ? input bytes differ between the API hash "
            "and the worker hash; ingesting under the local id.",
            result.video_id,
            video_id,
```

```
        )

    notify("ingesting (cloud_run)")
    storage_dict = (
        storage_cfg.model_dump()  # type: ignore[union-attr]  # hasattr-guarded; mypy can't
narrow
        if hasattr(storage_cfg, "model_dump")
        else {}
    )
    try:
        _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
    except Exception as e:  # noqa: BLE001 ? a missing/partial/corrupt result must not 500 or
leak rows
        logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
        return _failed(
            f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
            True,
        )

    # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then drops the
old ones.
    segments = [
        s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
    ]
    _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
    # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution` is the
real
    # Cloud Run execution name (cross-check: gcloud run jobs executions describe <execution>).
    provenance = {
        "path": "cloud_run",
        "requested": req.compute,
        "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
        "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
        "execution": result.execution,
        "outputs_uri": result.outputs_uri,
    }
    logger.info(
        "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s video_id=%s",
        result.execution,
        provenance["job"],
        provenance["region"],
        result.outputs_uri,
        video_id,
    )
    return (
        {
            "video_id": video_id,
            "status": "success",
            "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
            "results": [r.model_dump() for r in val_results],
            "play_segments": segments,
            "compute": provenance,
        },
        True,
    )


def _execute_processing(
    config=None, db=None, req=None, progress=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

    Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
    module-level ``global_config``/``db_instance`` (backward-compatible with existing
```

```
    tests that pass only ``req``). Route handlers always pass explicit params; older
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main  # call-time seam: main's globals/model stay patchable

    # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
    # shift positional args ? req arrives in position 0 (config), progress in position 1 (db).
    if config is not None and not isinstance(config, AppConfig):
        # Old signature: _execute_processing(req, progress=None)
        if req is None and db is not None and isinstance(db, _main.ProcessRequest):
            req = db
            db = None
        elif req is None and isinstance(config, _main.ProcessRequest):
            req = config
            config = None
    # Resolve config/db from module globals when not passed explicitly
    if config is None:
        config = _main.global_config
    if db is None:
        db = _main.db_instance

    notify = progress or (lambda stage: None)

    notify("hashing")
    # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged (identity ?
    # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The original
    # req.video_path (the source URI) is kept for the DB record + report; only the file-reading
    # consumers (hash / validators / segmenter) use the resolved local path.
    local_video = ""
    try:
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
        video_id = compute_video_id(local_video)
    except Exception:
        storage.cleanup_acquired_local_input(
            req.video_path, local_video, getattr(config, "storage", None)
        )
        raise
    try:
        existing_video = db.get_video(video_id)
    except Exception:
        storage.cleanup_acquired_local_input(
            req.video_path, local_video, getattr(config, "storage", None)
        )
        raise
    was_reingest = existing_video is not None

    # typed AppConfig: attribute access for the default segmenter (with safe fallback)
```

6. user (2026-07-02T09:20:59.636Z)

```
16:import uvicorn
178:    uvicorn surfaces a real bind failure loudly)."""
1690:        uvicorn.run(app, host=bind_host, port=port)
1693:if __name__ == "__main__":
1694:    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1696:    # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1700:    # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)
./docs/archives/checkpoints/2026-06-logging-reporting/README.md:12:- Logging setup in
`backend/main.py` and hot paths (motion, wasb_infer, stitcher, config loader, etc.).
```

./docs/archives/checkpoints/2026-06-logging-reporting/README.md:25:- Code: backend/reporting/, backend/main.py (logging.basicConfig), pipeline/segmenters/* (logger usage).
./docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:39:- Call sites in `backend/main.py` (both the CLI path and `_execute_processing` ~213/746) perform duck-typing: `isinstance(result, Failure)`, `hasattr(result, "value") else result`.
./docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:438: - The "parity" tests were not real: both paths were client.post("/api/process") (same job worker), and asserts were tautological (len==len on same expr). Must drive distinct: actual CLI-style `backend.main._execute_processing(req)` (or main() CLI) **and** job-worker (`_run_claimed_job` / drain) for same ProcessRequest; assert byte-identical DB rows/play_segments/reports (mod timestamps). Cover Failure + empty branches.
./docs/archives/CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md:486:- `backend/results.py`, `backend/config/models.py`, `backend/main.py`, `backend/pipeline/segmenters/base.py`
./docs/archives/decisions/DECISIONS.md:410:   `import backend.main` loads **no `training.*` module**.
./docs/archives/DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md:36:- Console entry points: `indexer = backend.main:main` (so `indexer --server` works),
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:31:- **Optional module globals** ? `db_instance` and `global_config` are `Optional[...] = None`, set only in `main()`. Importing `app` without `main()` ? NPE on every endpoint. The mypy quarantine on `backend.main` and `backend.api.job_worker` exists because of this.
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:47:| **D6** | **mypy quarantine ratchet:** when touching a quarantined module, try removing its `ignore_errors = True` entry from `mypy.ini`. | `mypy.ini` | D2 (app-factory) is the biggest opportunity ? if `backend.main` and `backend.api.job_worker` can leave quarantine, that's a major milestone. |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:71:| **D1** | HIGH | `backend/main.py` | **Massive god-module** (~1450 lines at audit; **2360 as of 2026-07-02** ? still growing). CLI, API, worker orchestration, upload, debug-clip, trim, reingest all in one file. | 2 |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:72:| **D2** | HIGH | `backend/main.py` + `backend/api/job_worker.py` | **Optional module globals** (`db_instance`, `global_config`). NPE-prone; mypy-quarantined. | **1** |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:75:| **D5** | MEDIUM | `backend/main.py` | `print()` used in production backend code (~20 calls in `--debug-clip`). Bypasses logging. | 3 |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:91:| **O2** | MEDIUM | `backend/main.py` | Single `ThreadPoolExecutor(max_workers=1)` ? one stuck job blocks all. | 2 |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:107:| **Q8** | `backend/main.py` | Add comment explaining lazy `import cv2` in `--debug-clip` (cv2 not in LIGHT tier). | 1 |
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:155:- `backend/main.py` ? FastAPI app definition, all endpoints
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:156:- `backend/api/job_worker.py` ? resolves `db_instance`/`global_config` through `import backend.main as _main`
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:158:- `tests/test_async_jobs.py` ? already monkeypatches `backend.main`
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:162:**Test strategy:** All existing tests must pass byte-identically. The `TestClient` usage changes from `from backend.main import app` to `from backend.main import create_app; app = create_app(test_config)`.
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:223:- [x] P5: `create_app()` injects shared state via `app.state` (handlers read `request.app.state`); `backend.api.job_worker` removed from mypy quarantine. (The Optional module globals `db_instance`/`global_config` are **retained as a transitional back-compat shim** and `backend.main` stays mypy-quarantined ? full removal is follow-up, see D1/P10.)
./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:281:- `2026-06-25` ? **Accuracy fixes (docs-only).** (1) §8 DoD P5 said the module globals were "replaced with `app.state`"; corrected ? `create_app()` *added* `app.state` and lifted `job_worker` from quarantine, but `db_instance`/`global_config` remain as a transitional shim and `backend.main` is still mypy-quarantined (`backend/main.py:264`, `mypy.ini`). (2) `main.py` size "~1300" ? "~1450" (now 1448 lines) in §3b/§6/§8.1.

./docs/archives/past_projects/CODE_CLEANUP_AUDIT_2026-06-24-SUPERSEDED-2026-07-02.md:285:-
`2026-07-02` ? **P10 currency + anchor fix (docs-only, audit-reconcile).** Re-measured
`backend/main.py`: **2360 lines** (1448 on 2026-06-25 ? the endpoint/fix PRs since keep landing
here: `/api/process` idempotency, remote-source caching, ffmpeg timeouts, scratch cleanup,
config-correctness fixes) ? refreshed in §3b/§6/§8.1. Recorded that the step-1 extraction PR
#390 (`backend/orchestration.py`, cut 2026-06-26) is stale ? it predates the merged commits
landed since (a 1448?2360 growth of `main.py`) ? and should be re-cut fresh from
`origin/master`. Also corrected the tracking anchor (the tracker is **§6**, not §7) in the
header and `docs/README.md`. No code change.
./docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md:88:| Everything
assumes a **seekable local file**: `cv2.VideoCapture`, `ffmpeg -i <path>`, whole-file MD5, an
`allowed_video_root` path check ([`main.py`](../../backend/main.py)) | **MEDIUM.** No URL/stream
path. Needs a download-to-local shim + output upload. (Sync-first copy to NVMe is the right
shape.) |
./docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md:30: check in
[`main.py`](../../backend/main.py). No URL/stream path ? needs a `fetch(uri)?local` + `put`
shim.
./docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md:80:|
`R1-01` main.py line count | Still open, number changed | `backend/main.py` is now **2360
lines** (was ~1965 when this plan was bootstrapped; the Wave-1 endpoint/fix PRs since land
here), still large; `backend/orchestration.py` absent; PR `#390` remains open/stale (tracked as
`CODE_CLEANUP_AUDIT` P10 ? re-cut fresh from `origin/master`). |
./docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md:144:|
Engine behavior fixes conflict with stale `#390` | `backend/main.py` | Land correctness/resource
fixes first, then close/re-cut the split from fresh `origin/master`. |
./docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md:187:|
P24 | 5 | Remove remaining `backend.main` shim cycles and lift mypy quarantine where type-clean.
| engine | P23 | No | Todo | ? |
./docs/archives/past_projects/KHELSUTRA_GURU_HEALTH_REMEDIATION_PLAN-FOLDED-2026-07-02.md:228:-
Engine job path: `backend/main.py`, `backend/api/job_worker.py`,
`backend/infrastructure/database.py`
./docs/archives/PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md:51: - The CLI `--output-dir`
(already in `backend/main.py`) becomes the **parent**; the pipeline
./docs/archives/TEST_RUN_SUMMARY.md:60:   python -u -m backend.main --video-path
"C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-frames 500
./docs/AUDIT_REPORT_khelsutra-guru_2026-07-02.md:16:The engine is a ~2200-line god-module
orchestrator (backend/main.py) over a well-factored worker/storage/compute substrate; master is
current (HEAD e410136) and CI is trusted green with ~1500 tests. The async job plumbing
(claim_due_job CAS + single-worker executor + single-instance DB lock) and the schema ladder
(v9, idempotent ALTERs, backup-before-migrate, downgrade guard) are genuinely solid ? most
crash-safety patterns hold up under scrutiny. The dominant hot-path debt is lifecycle, not
logic: paid-segmenter idempotency is still missing (#340 open, confirmed at HEAD), remote-input
scratch copies are never reclaimed anywhere (acquire_local_input mkdtemp-per-call, worst on
/api/source where every browser Range request re-downloads the full remote video), and the
packaged-app path story is inconsistent (upload_dir CWD-relative vs app-home-rooted output_dir;
config output.output_dir dead on the CLI path). Cloud-serving has an owner-gated GPU cost leak
(poll timeout never cancels the Cloud Run execution). Findings are 2×P1, 5×P2, 2×P3; only #340
is already tracked.

7. user (2026-07-02T09:21:09.609Z)

```
        # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
        _get_executor().submit(_drain_due_jobs, global_config, db_instance)
        # Stamp shared state on the app so route handlers access it via request.app.state
        # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
        create_app(global_config, db_instance)
        uvicorn.run(app, host=bind_host, port=port)


if __name__ == "__main__":
    # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads` resolve
the
    # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ? a
SEPARATE
    # module object. If `main()` runs here it configures `__main__`'s globals, leaving
```

```
    # `backend.main.db_instance` = None; the async-job drain then crashes silently on
    # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch through
    # `backend.main` so `main()` configures the module everyone actually reads. (No recursion:
    # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-fire.)
    import backend.main as _bm

    _bm.main()
import argparse
import json
import logging
import os
import shutil
import socket
import sys
import tempfile
import threading
import uuid
import webbrowser
from importlib.metadata import PackageNotFoundError
from importlib.metadata import version as _pkg_version
from urllib.parse import quote

import uvicorn
from dotenv import load_dotenv
```

PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) ke exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_dotenv stdlib-only import ? keeps this pre-pydantic line light.

```
from backend.utils.app_home import env_file_to_load, resolve_output_dir

load_dotenv(env_file_to_load())
```

Centralized logging setup for the indexer (hot paths use logger, CLI output uses print

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger(__name__)
from typing import Any, Dict, List, Optional

from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from starlette.middleware.trustedhost import TrustedHostMiddleware

from backend import (
    billing,  # M8: per-job cost ledger (USD internal / INR display)
    storage,  # M2: URI-aware input acquisition (local = identity passthrough)
)
from backend.api import (
    auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
)
from backend.config import (
    AppConfig,
    StartupCheckError,
    enforce_startup_checks,
    write_light_default_config,  # P2b: batteries-included first-run config seeding
```

```
)
from backend.config import (  # new typed config
    load_config as load_app_config,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.validators.base import registry
from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure  # standardized error/result contract
from backend.storage.capabilities import (
    storage_capabilities,  # P3: secret-free "which media can this box use" report
)
from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffprobe_bin,
)
from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import (
    output_base,
    resolve_under_root,
    safe_output_filename,
    validate_input_path,
)
```

8. user (2026-07-02T09:21:23.099Z)

```
301:# dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestration
306:from backend.orchestration import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
275:db_instance: Optional[Database] = None
276:global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
328:class ProcessRequest(BaseModel):
```

9. user (2026-07-02T09:21:30.694Z)

```
    _maybe_record_job_cost,
    _run_claimed_job,
    _schedule_next_drain_if_waiting,
)
```

The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, C dispatch+ingest, compile resolution + worker handler) now lives in backend.orchestra (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressab backend.main ? `import backend.main as m; m._execute_processing(...)`, the job work call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main, all keep working unchanged.

```
from backend.orchestration import (  # noqa: F401  (re-exports: names stay addressable on
backend.main)
    _cloud_available,
    _CompileError,
    _execute_compile,
    _execute_processing,
    _finalize_reingest,
    _ingest_remote_segments,
    _maybe_dispatch_remote,
    _remote_input_size_bytes,
    _report_config_for_input,
```

```
    _resolve_compile,
)
```

Legacy thin wrapper for any remaining direct calls during transition.
New code should import load_app_config / AppConfig from backend.config directly.

```
def load_config(config_path: str = "config.json") -> Dict[str, Any]:
    cfg = load_app_config(config_path)
    return cfg.model_dump(mode="json")
```

--- API Models ---

```
class ProcessRequest(BaseModel):
    video_path: str
    player_pool: Optional[List[str]] = None
    max_frames: Optional[int] = None
    segmenter: Optional[str] = None
    # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job for PMF
    # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older clients).
    locale: Optional[str] = None
    # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured server;
    # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ? server
    # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
    compute: Optional[str] = None
    # Explicit opt-in reprocess path. By default, a video whose content hash already has stored
    # play_segments is returned from cache so refresh/Try again cannot re-bill paid segmenters.
    force: bool = False


class AnnotationsRequest(BaseModel):
```

10. user (2026-07-02T09:21:44.366Z)

```
tests/test_api_annotations.py:91:    monkeypatch.setattr(m, "_remote_input_size_bytes", lambda
*a, **k: len(payload))
tests/test_api_assets.py:129:    monkeypatch.setattr(m, "_remote_input_size_bytes", lambda *a,
**k: len(payload))
tests/test_api_cloud_dispatch.py:128:    """Make backend.compute.get_compute_backend (re-
imported inside _maybe_dispatch_remote) build a
tests/test_api_cloud_dispatch.py:151:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:170:    m._execute_processing(
tests/test_api_cloud_dispatch.py:196:    m._execute_processing(
tests/test_api_cloud_dispatch.py:209:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:228:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:269:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:298:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:312:        out = m._execute_processing(
tests/test_api_cloud_dispatch.py:353:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:396:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:427:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:445:        out = m._execute_processing(
tests/test_api_cloud_dispatch.py:457:def test_capabilities_reports_cloud_available(cloud_env):
tests/test_api_cloud_dispatch.py:602:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:624:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:648:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:676:    out = m._execute_processing(
tests/test_api_cloud_dispatch.py:704:    out =
m._execute_processing(m.ProcessRequest(video_path=str(video)))
tests/test_api_endpoints.py:815:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
len(payload))
tests/test_api_endpoints.py:851:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k: 10
* 1024**3)
```

```
tests/test_api_endpoints.py:868:    mp.setattr(m, "_remote_input_size_bytes", lambda *a, **k:
None)
tests/test_async_jobs.py:489:def test_worker_dispatches_compile_kind_to_execute_compile(env):
tests/test_async_jobs.py:490:    """The claimed-job dispatcher routes kind='compile' to
_execute_compile (NOT _execute_processing),
tests/test_async_jobs.py:503:    mp.setattr(m, "_execute_compile", _fake_compile)
tests/test_main_module_budget.py:41:    "_maybe_dispatch_remote",
Binary file tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc matches
```

11. user (2026-07-02T09:21:55.381Z)

```
backend/main.py:314:    _remote_input_size_bytes,
backend/main.py:643:        need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage",
None))
backend/main.py:872:            need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage",
None))
backend/main.py:1017:        need = _remote_input_size_bytes(
backend/orchestration.py:593:def _remote_input_size_bytes(input_ref, storage_config) ->
Optional[int]:
backend/api/job_worker.py:9:every cross-function seam ? `db_instance`, `global_config`,
`_execute_processing`,
backend/api/job_worker.py:60:    is the segmenter's `_execute_processing`, run UNDER the
destroy-after-confirm contract.
backend/api/job_worker.py:114:            result = _main._execute_compile(
backend/api/job_worker.py:122:            result = _main._execute_processing(
backend/main.py:309:    _execute_compile,
backend/main.py:310:    _execute_processing,
backend/main.py:316:    _resolve_compile,
backend/main.py:1170:        video, _kept, out_name = _resolve_compile(
backend/orchestration.py:3:The full hash -> validate -> segment -> report pipeline
(``_execute_processing``), the
backend/orchestration.py:5:and the shared compile resolution + worker handler
(``_resolve_compile`` / ``_execute_compile``).
backend/orchestration.py:350:    test code that calls ``_execute_processing(req)`` still works.
backend/orchestration.py:686:        video, kept, out_name = _resolve_compile(
```

12. user (2026-07-02T09:22:09.646Z)

```
---
tests/test_api_annotations.py:24
tests/test_api_annotations.py:91
tests/test_api_assets.py:129
tests/test_api_cloud_dispatch.py:102
tests/test_api_cloud_dispatch.py:103
tests/test_api_cloud_dispatch.py:112
tests/test_api_cloud_dispatch.py:113
tests/test_api_cloud_dispatch.py:114
tests/test_api_cloud_dispatch.py:240
tests/test_api_cloud_dispatch.py:241
tests/test_api_cloud_dispatch.py:243
tests/test_api_cloud_dispatch.py:244
tests/test_api_cloud_dispatch.py:245
tests/test_api_cloud_dispatch.py:372
tests/test_api_cloud_dispatch.py:381
tests/test_api_cloud_dispatch.py:382
tests/test_api_cloud_dispatch.py:414
tests/test_api_cloud_dispatch.py:418
tests/test_api_cloud_dispatch.py:547
tests/test_api_cloud_dispatch.py:556
tests/test_api_cloud_dispatch.py:557
tests/test_api_cloud_dispatch.py:637
tests/test_api_cloud_dispatch.py:638
tests/test_api_cloud_dispatch.py:639
tests/test_api_cloud_dispatch.py:658
tests/test_api_cloud_dispatch.py:659
```

```
tests/test_api_cloud_dispatch.py:694
tests/test_api_cloud_dispatch.py:695
tests/test_api_endpoints.py:32
tests/test_api_endpoints.py:93
tests/test_api_endpoints.py:305
tests/test_api_endpoints.py:345
tests/test_api_endpoints.py:411
tests/test_api_endpoints.py:455
tests/test_api_endpoints.py:486
tests/test_api_endpoints.py:531
tests/test_api_endpoints.py:575
tests/test_api_endpoints.py:782
tests/test_api_endpoints.py:815
tests/test_api_endpoints.py:817
tests/test_api_endpoints.py:851
tests/test_api_endpoints.py:868
tests/test_api_endpoints.py:1016
tests/test_api_endpoints.py:1017
tests/test_api_endpoints.py:1018
tests/test_api_endpoints.py:1020
tests/test_api_endpoints.py:1066
tests/test_api_endpoints.py:1067
tests/test_api_endpoints.py:1068
tests/test_api_endpoints.py:1070
```

13. user (2026-07-02T09:22:19.212Z)

```
     15 setattr(m, "global_config"
     15 setattr(m, "db_instance"
     10 setattr(m, "_get_executor"
      5 setattr(m, "VideoCompiler"
      5 setattr(m, "_remote_input_size_bytes"
      2 setattr(m, "load_app_config"
      2 setattr(m, "Database"
      2 setattr(m, "_arm_wake_timer"
      1 setattr(m, "emit_indexer_report"
      1 setattr(m, "compute_video_id"
      1 setattr(m, "_execute_compile"
```

14. user (2026-07-02T09:22:36.418Z)

```
== orchestration usage ==
29:from backend.pipeline.stitcher.compiler import VideoCompiler
32:from backend.reporting import emit_indexer_report
34:from backend.utils.hashing import compute_video_id
385:        video_id = compute_video_id(local_video)
570:        paths = emit_indexer_report(
699:    compiler = VideoCompiler(
== main usage ==
61:from backend.pipeline.stitcher.compiler import VideoCompiler
64:from backend.reporting import emit_indexer_report
74:from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
359:    # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
822:    MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
891:        video_id = compute_video_id(local_working)
1437:        compiler = VideoCompiler(
1559:            video_id = compute_video_id(local_video)
1621:                paths = emit_indexer_report(
```

15. assistant (2026-07-02T09:23:51.149Z)

```
API Error: Connection closed mid-response. The response above may be incomplete.
```